

S. SUPPLEMENTARY MATERIAL

S.1. Training setup

NER was framed in this study as a multilabel token classification task. Each dataset was split into training, validation, and test datasets, following the proportions of 60%, 20%, and 20%. Two different techniques for splitting data were tested and compared: simple randomization and iterative data stratification.

Multilabel datasets often suffer from severely imbalanced classes, which diminishes the capabilities of any algorithm that learns from the data. Splitting the data while preserving the representation of rare classes is an important but often overlooked step. The iterative stratification algorithm (1,2) prioritizes the distribution of rarer classes while taking into account how a particular class is distributed in combination with other classes. We used the implementation available at scikit-multilearn (3).

Two entity annotation schemes were evaluated in this study: **IO**, in which tokens can receive an inside tag “I” representing a class, or an outside “O” tag representing no class; and **BIO**, which also includes a beginning “B” tag for each class.

The training of each BERT model was implemented in Python using the *transformers* library (4). For consistency, the same hyperparameters were used whenever possible: 0.00005 as the learning rate, 10 as the batch size, 5 for gradient accumulation, and a maximum input length of 512. The metric used for the best model selection was the micro F1 score on the validation set. Training was stopped early with a patience parameter of 5. The loss function was the binary cross entropy loss. Different training parameters were used in case a model failed to converge. In the breast cancer dataset, a learning rate of 0.0001 and a patience of 15 were used.

In order to account for class imbalance, unweighted and weighted losses were compared. Class weights were computed as suggested in the PyTorch documentation (5). The weight w_c for a class c was defined as $w_c = \frac{N_{c'}}{N_c}$, where $N_{c'}$ is the number of clinical notes not containing class c entities, and N_c is the number of clinical notes containing class c entities.

Oversampling was also explored to tackle class imbalance. The oversampling algorithm replicated examples in the training dataset, prioritizing the less frequent classes until their frequency was similar to the average class frequency.

For reproducibility, the source code of this study was made available in the following GitHub repository: https://github.com/GRUPOMED4U/clinical_ner_benchmark_paper

S.2. Hardware

This study was conducted on a desktop equipped with a 13th Gen Intel(R) Core(TM) i9-13900K (3.00 GHz, 24 cores), 64 GB DDR5 RAM, and an NVIDIA GeForce RTX 4090 GPU with 24 GB VRAM. The system had a 4 TB Kingston SSD for storage and Windows 11 Pro as the operating system.

Table S.1. Prompt strategy used for entity extraction in clinical notes using LLMs. The fields defined with curly braces were replaced by the respective entities, text and JSON object expected as the response.

Role	Prompt
system	Extract the following entities from the given clinical note in order of appearance. Use exact text for extractions. Do not paraphrase or overlap entities. The entities to extract are: {entities}. See the examples below for the expected format. # Clinical note example: {example_text} # Ideal response example: {example_response}
user	# Clinical note: {input_text}

Source: Authors.

Table S.2. Resource usage of the gpt-5 model in each reasoning effort level. The measures of time, token usage and cost represent the average across all responses. Time is measured in seconds. Cost is estimated per 1000 responses and is presented in USD considering the pricing at November 14th, 2025. Token usage refers to the total number of tokens (prompt and completion tokens).

Effort level	Time	Token usage	Cost
Minimal	18.3	3064	9.57
Low	81.4	6090	39.83
Medium	115.5	10,508	84.01

Source: Authors.

Table S.3. Comparison of methods to deal with imbalanced classes using mmBERT-base model. For this comparison, both datasets were used with the IO data annotation scheme. Training for each data split method was repeated for 10 iterations. The mean and standard deviation (std) of the micro and macro F1-score are reported. Clamped weights, in this context, mean fixing the minimum weight for the classes at 1.0.

Methods	Macro F1-score ($\pm$ std)	Micro F1-score ($\pm$ std)
SemClinBr		
Random data split	0.6567 ($\pm$ 0.0200)	0.6904 ($\pm$ 0.0287)
IS (Iterative Stratification)	0.7139 ($\pm$ 0.0050)	0.7646 ($\pm$ 0.0094)
IS with weighted loss	0.6672 ($\pm$ 0.0094)	0.6773 ($\pm$ 0.0140)
IS with weighted loss and clamped weights	0.7118 ($\pm$ 0.0125)	0.7604 ($\pm$ 0.0202)
IS with OS (oversampling)	0.7154 ($\pm$ 0.0117)	0.7634 ($\pm$ 0.0073)
Breast cancer dataset		
IS	0.6928 ($\pm$ 0.0290)	0.7301 ($\pm$ 0.0199)
IS with weighted loss	0.7283 ($\pm$ 0.0091)	0.7591 ($\pm$ 0.0036)
IS with weighted loss and clamped weights	0.7340 ($\pm$ 0.0115)	0.7634 ($\pm$ 0.0079)
IS with OS (oversampling)	0.7140 ($\pm$ 0.0110)	0.7584 ($\pm$ 0.0106)

Source: Authors.

REFERENCES

1. Sechidis K, Tsoumakas G, Vlahavas I. On the Stratification of Multi-label Data. Machine Learning and Knowledge Discovery in Databases [Internet]. 2011 [cited 2026 Mar 25];145–58. Available from: https://link.springer.com/chapter/10.1007/978-3-642-23808-6_10
2. Szymański P, Kajdanowicz T. A Network Perspective on Stratification of Multi-Label Data [Internet]. 2017 [cited 2026 Mar 25]. Available from: <http://arxiv.org/abs/1704.08756>
3. scikit-multilearn [Internet]. [cited 2026 Mar 25]. Available from: <https://github.com/scikit-multilearn/scikit-multilearn>
4. Transformers · Hugging Face [Internet]. [cited 2026 Mar 25]. Available from: <https://huggingface.co/docs/transformers/index>
5. BCEWithLogitsLoss — PyTorch 2.11 documentation [Internet]. 2023 [cited 2026 Mar 25]. Available from: <https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html>